

정우태 포트폴리오 초안

출시 경험, 성능 재설계, AI 협업 구조화, 팀 프로젝트
시스템 구현을 중심으로 정리한 PDF 초안입니다.

Unity / C#

모바일·2D·VR 클라
이언트 개발

Released

Google Play, itch.io
배포 경험

AI Workflow

AI 작업 규칙·검증
루프 설계



게임을 끝까지 만들고, 출시 후 구조를 다시 보는 개발자

단순 구현 목록보다 "문제를 어떻게 좁히고, 어떤 구조로 다시 설계했으며, 어떻게 검증했는가"를 보여주는 방향으로 구성했습니다. 이 초안은 기존 웹 포트폴리오의 텍스트, GIF, 링크를 기반으로 하되 PDF용으로 밀도를 조정했습니다.

5+

대표 프로젝트

2

공개 배포 경로: Google Play /
itch.io

2026.05

Unity Certified Associate: Game
Developer

핵심 포지션

- Unity / C# 게임 클라이언트 개발자
- 모바일 클라이언트와 2D 게임 구조 설계 지향
- 성능 문제를 프로파일링하고 구조를 다시 잡는 작업에 강점
- AI를 코드 생성기가 아니라 작업 프로세스의 구성원으로 다루는 방식 실험

PDF에서 증명할 역량

- 출시/배포: Ghost March, Cubika
- 성능 최적화: Ghost March DOTS 재설계
- 아키텍처: Match3LogueLike asmdef / EventBus
- 트러블슈팅: LostCone Input System, Ghost March UI/GPU
- 팀 협업: IronSarcophagus 역할 경계와 시스템 구현

MyTokenMate는 공개 GitHub 링크는 제공할 수 있지만, 현재 포트폴리오 본문에서는 "AI 작업량을 추적하려 만든 개인 생산성 도구" 정도로만 짧게 언급하는 편이 적절합니다.

Ghost March: 출시작을 기준으로 두고 DOTS 구조로 재설계



Unity

C#

ShaderLab

Google Play

Solo

Ghost March는 공모전 수상과 Play Store 출시 경험이 있는 대표 프로젝트입니다. 초안에서는 "5주 제작"보다 출시 이후 Legacy 구조를 보존한 채 DOTS 기반 수직 슬라이스로 재설계한 과정과 실측 수치를 강조합니다.

Play Store 열기

Gameplay 영상 열기

문제 인식

- `GameManager.instance`가 상태, 입력, UI, 저장, 일시정지를 폭넓게 소유.
- `Weapon.Update()` switch 기반 무기 분기로 확장 비용 증가.
- PoolManager prefab 배열 순서가 런타임 id 계약 처럼 쓰여 씬과 코드가 결합.
- 대량 적, 데미지 텍스트, UI Toolkit 갱신으로 프레임 비용 증가.

재설계 판단

- Legacy MonoBehaviour 구조를 비교 기준으로 격리.
- Enemy, Bullet, Soul/Drop, Wave, Damage Event를 DOTS 수직 슬라이스로 재구현.
- ScriptableObject 설정을 ECS Blob으로 변환.
- MonoBehaviour는 UI, 입력, 씬 부트스트랩, 외부 서비스 연동에 제한.

9 fps

Legacy 1500 enemies

67 fps

DOTS 4096 enemies

**109.5ms ->
14.92ms**

CPU frame time

Ghost March: 성능, 렌더링, 운영 기능까지 이어진 검증

트러블슈팅 타임라인

GPU Instancing 전환 후 플레이어/적/소울 소실. stale serialized scene value와 snapshot fallback 부재를 분리해 추적.

Separation EnemySeparation 더블버퍼에서 절대 좌표 기록이 chase 이동을 덮어씀. `PositionDelta` 방식으로 해결.

RenderMesh RenderMeshIndirect 도입 후 GPU 비용 폭증. URP 2D 궁합 문제로 DrawMeshInstanced 유지 판단.

UI Toolkit 매 프레임 label.text 업데이트가 패널 전체 re-render를 유발. 변경 감지로 dirty 빈도 감소.

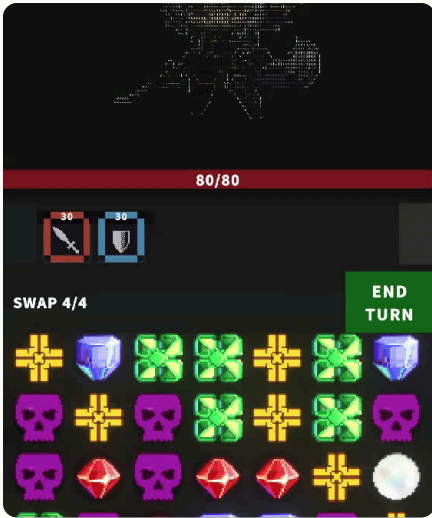
실측 개선

- UI Toolkit 최적화: GPU 16.93ms -> 11.22ms.
- UI Toolkit 최적화: CPU 17.47ms -> 11.43ms.
- Instanced batches: 691 -> 약 4 batches.
- TMP 한글 깨짐은 glyph 범위, dynamic source font, sampling size 이슈로 분리.

운영/서비스 확장

- AdMob, IAP/remove ads, localization, app update prompt 문서화.
- Android release AAB: versionCode, manifest, Billing permission, Play Billing metadata 검증.
- Unity IAP / Play App Update 런타임 클래스가 R8로 제거되는 문제를 keep rule로 대응.
- DOTS gameplay assembly에 Play Core 의존성을 넣지 않는 boundary 명시.

Match3LogueLike: AI가 구조를 지키도록 설계한 프로젝트



Unity 6

URP 2D

AI Workflow

asmdef

Tests

이 프로젝트의 핵심은 AI 사용 여부가 아니라 AI가 안전하게 작업할 수 있도록 프로젝트 구조, 규칙, 테스트, 작업 lane을 설계한 점입니다.

[Gameplay 영상 열기](#)

의존성 방향

Match3.Data

->

Match3.Core

->

Board

Combat

Roguelike

Meta

UI

Ads

Board와 Combat의 직접 참조는 금지하고 EventBus를 통해서만 통신하도록 설계했습니다. Data 영역은 MonoBehaviour를 금지하고 SO/interface/enum 중심으로 유지합니다.

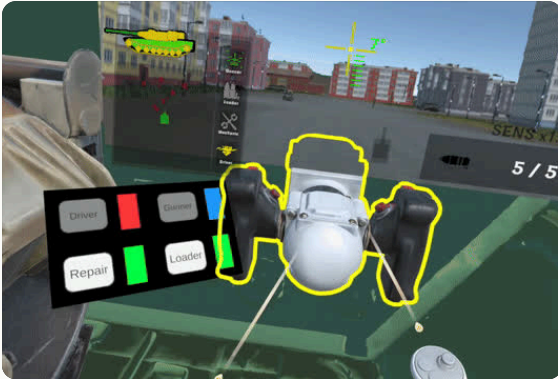
AI 협업 설계

- 범용 49-agent 템플릿이 아니라 프로젝트 모듈과 1:1 대응하는 8개 agent 설계.
- arch-guardian, board-agent, combat-agent, ui-agent, data-agent 등 역할 분리.
- Data MonoBehaviour 금지, Find 금지, Update 할당 금지, god object 감지 hook.
- Coordinator / Worker / Verify 역할을 분리하고 worker별 worktree와 lane ownership 운영.

검증 근거

- C# 테스트 파일 34개.
- RewardOverlayTests PlayMode 10/10 succeeded.
- SaveManagerPhase1SmokeTests EditMode 4/4 passed.
- UI/UX visual QA에서 "기능 테스트 통과와 실제 화면 가독성은 다르다"는 residual risk를 별도 기록.

IronSarcophagus: 2인 VR 팀 프로젝트에서 맡은 시스템 경계



VR

XR Toolkit

Team 2

Network

Enemy AI

팀 프로젝트이므로 본인 담당 범위를 분명히 쓰는 것이 중요합니다. 본인 범위는 네트워크 기반 구조, 포수 VR 조준 인터랙션, BallisticCalc, FireSystem, 적 AI, 게임플레이 모드로 정리합니다.

[Gameplay 영상 열기](#)

포수/탄도 시스템

AimController

->

BallisticCalc

->

FireSystem

->

TankStatus

- 포물선 탄도 시뮬레이션과 TimeOfFlight 계산.
- Linecast 기반 착탄과 시각 발사체/피해 판정 위치 일치.
- OnImpact 이벤트로 VFX/SFX 연동 경계 제공.

적 AI와 피해 판정

- 총구 기준 5개 포인트 Linecast로 플레이어 노출도 계산.
- 20% 미만 노출은 차폐 상태로 간주.
- 12방향 후보 중 NavMesh + 시야 확보 가능한 재 배치 위치 탐색.
- NetworkVariable 기반 부위별 HP, 장갑 두께, 관통, 부위 파괴 처리.

UI/HUD, 장전, 운전, 수리 등 팀원 담당 영역은 본인 구현처럼 쓰지 않고, 역할 경계와 인터페이스 합의의 근거로만 사용합니다.

LostCone: 공개 개발과 간헐적 Input System 버그 해결



Unity

DOTween

Aseprite

User Feedback

Troubleshooting

LostCone은 소재 자체보다 유저층 이해, 공개 개발, 플레이테스트, 엔진 상태까지 추적한 트러블슈팅 사례로 배치하는 것이 좋습니다.

Gameplay 영상 열기

공개 개발일지 열기

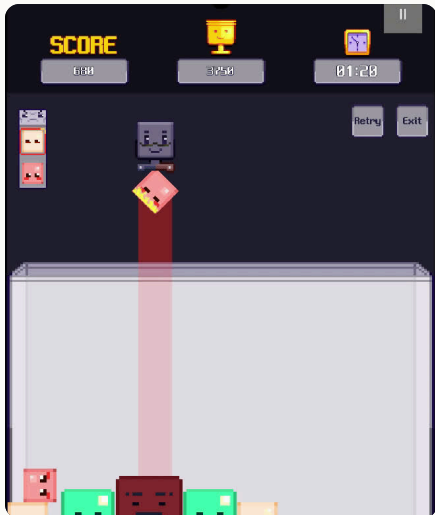
공개 개발 사이클

- 개발일지 50회, 플레이테스트 3회, 커뮤니티 피드백 수집.
- 처음에는 퍼즐 요소가 있는 메트로배니아로 기획.
- 실제 유저층이 가수 팬층이라는 점을 고려해 이스터에그를 찾는 힐링 게임 방향으로 전환.
- Aseprite 리소스 직접 제작, DOTween 기반 연출 구성.

Input System 트러블슈팅

- 증상: 같은 코드/커밋에서도 키보드 입력이 간헐적으로 중단.
- 실패한 시도: ActionMap 강제 활성화, polling 전환, sceneLoaded 대응, Domain Reload 대응, Library 삭제.
- 원인: 터치스크린 노트북에서 Control Scheme이 Touch로 auto-switch.
- 해결: Keyboard&Mouse scheme 강제, prefab default scheme 변경, auto-switch 해제.

Cubika와 기타 보조 프로젝트



Unity

WebGL

itch.io

Puzzle

Mobile Porting

Cubika는 작은 프로젝트지만, WebGL 배포와 출시 후 모바일 조작, 게임오버, 광고/IAP scaffold로 확장한 기록이 있어 "작게 완성하고 배포한 경험"으로 유효합니다.

[itch.io 플레이 열기](#)
[Gameplay Shorts 열기](#)

프로젝트	포트폴리오에서의 역할	넣을 내용
Cubika	작은 출시작 / 배포 경험	큐브 회전·낙하·밀기, 3초 game-over grace, 모바일 조작, Ads/IAP scaffold, itch.io WebGL.
Dong_Mae	기타 구현 경험	타일맵, 문/로프/점프패드/함정/물/숨겨진 벽, 보스 패턴, 카메라-페이드 연출. Appendix 표로 충분.
MyTokenMate	AI workflow 보조 사례	Claude/Codex/Gemini/Copilot 로컬 로그를 읽는 Electron overlay. 공개 repo는 있으나 본문에서는 짧게 언급.
_dashboard	직접 제작물 아님	외부 도구를 가져와 프로젝트 관리에 적용한 사례로만 제한. 별도 프로젝트 페이지로 쓰지 않음.

AI를 쓰는 방식: 생성보다 구조, 검증, 책임 경계

면접 복기와 저장소 문서에서 반복적으로 보이는 강점은 "AI를 쓴다"가 아니라, AI가 실수해도 프로젝트가 깨지지 않도록 규칙과 검증 루프를 설계한다는 점입니다.

AI 협업 원칙

- 넓은 단위 작업은 Codex, 좁은 범위 검증과 리뷰는 Claude Code로 분리.
- Unity MCP를 연결해 에디터 상태 확인과 검증을 작업 흐름에 포함.
- 작업 lane과 파일 소유권을 나누어 scene/prefab/SO 충돌을 줄임.
- agent, skill, hook을 프로젝트 구조에 맞게 재구성.

검증 루프

- 기능 테스트와 실제 화면 QA를 별도 체크로 다룸.
- architecture scan, path guard, git diff check, merge preview를 agent evidence로 기록.
- AI가 만든 결과물은 테스트, 빌드, visual QA, 문서 일치 여부로 검증.
- 토큰/컨텍스트 압축 도구는 사용자 리뷰와 프로젝트 적합성을 확인 후 적용.

MyTokenMate 언급 수준

AI coding tool 사용량을 작업 중 확인하려 만든 개인 Electron 도구. Claude/Codex의 raw token 합산 한계를 고려해 rate limit event/statusLine cache를 우선 표시하려는 접근은 좋지만, 아직 테스트/빌드/릴리즈 근거가 약해 별도 프로젝트 페이지보다 보조 사례가 적합합니다.

커뮤니케이션 사례

- 대학 조별과제에서 첫 회의 침묵이 길어질 때 먼저 안건을 정리하고 역할을 나눔.
- 영상 팀에는 콘티, 대본 팀에는 초안을 제공해 협업 진입 장벽을 낮춤.
- 문제를 한쪽 잘못으로 단정하기보다 회색 지점을 찾는 태도로 설명 가능.

기술 스택과 공개 링크

주요 기술

Unity C# UGUI UI Toolkit URP 2D
 DOTween ShaderLab HLSL Aseprite
 DOTween XR Toolkit Netcode Electron
 React TypeScript

자격/기타

- Unity Certified Associate: Game Developer (2026.05)
- 2023 충청권 게임 인디유 공모전 장려상
- OPIc IM2, 일본어 일상회화
- 지원 방향: Unity / C# 게임 클라이언트, 모바일 클라이언트

대상	링크	PDF 사용
GitHub	https://github.com/NulMa	Contact 대표 링크
Ghost March	Play Store	최우선 공개 결과물
Cubika	itch.io	작은 출시작
LostCone	개발일지	공개 개발 근거
MyTokenMate	GitHub public repo	보조 언급. 별도 페이지 비추천
Email	jungwt524@gmail.com	연락처

다음 버전에서는 각 프로젝트별 대표 코드 2~3개와 다이어그램을 추가하면 PDF의 설득력이 더 커집니다. 특히 Ghost March의 DOTween 경계, Match3의 EventBus 흐름, IronSarcophagus의 탄도/노출도 계산은 코드 스니펫과 잘 맞습니다.